Home: https://github.com/nvim-treesitter/nvim-treesitter

Treesitter
Home: https://github.com/tree-sitter/tree-sitter
A generic parser to provide better **syntax** related support
e.g. syntax highlighting
e.g. folding

nvim-treesitter
A neovim plugin for using treesitter in neovim.
A replacement of the original vim's **regex based** syntax features by using a real **parser**.
e.g. syntax highlighting
See **neovim cheatsheet** for details.

Features are called **modules**.
e.g. highlight module for syntax highlighting

IMPORTANT: plugin was refactored for neovim 0.12
Old (<= neovim 0.11): https://github.com/nvim-treesitter/nvim-treesitter/tree/master
New (>= neovim 0.12): https://github.com/nvim-treesitter/nvim-treesitter/tree/main

Language parser
Languages: https://github.com/nvim-treesitter/nvim-treesitter#supported-languages
To apply treesitter on a file, a **parser** needs to be generated.
Each language requires its own parser i.e. a **language parser**.
e.g. a javascript parser

Modules
Built-in modules: https://github.com/nvim-treesitter/nvim-treesitter#supported-languages
Other modules: https://github.com/nvim-treesitter/nvim-treesitter/wiki/Extra-modules-and-plugins

Modules
- **highlight**: syntax highlighting
e.g.
Without highlight

```
startServer :: ServerSettings -> IO ()
startServer settings = bracket (uninterruptibleMask_ setup) (uninterruptibleMask_
  where
    setup = do
      (fileLogger, handler) <- createDefaultFileLogger
      shutdownVar <- createShutdownHandler
      return (fileLogger, handler, shutdownVar)
    start (fileLogger, _, shutdownVar) = startServer' (settings, N.defaultSetting
    cleanup (_, h, _) = hClose h
```

With highlight

```haskell
startServer :: ServerSettings -> IO ()
startServer settings = bracket (uninterruptibleMask_ setup) (uninterruptibleMask_
  where
    setup = do
      (fileLogger, handler) <- createDefaultFileLogger
      shutdownVar <- createShutdownHandler
      return (fileLogger, handler, shutdownVar)
    start (fileLogger, _, shutdownVar) = startServer' (settings, N.defaultSetting
    cleanup (_, h, _) = hClose h
```

- **indent**: smart indenting
- **incremental_selection**: visually select incrementally by parser node

Initial selection is node under cursor.
Node increment moves up one parent
e.g. quotes ''
e.g. brackets ()
Scope increment moves up one scope
e.g. function scope

e.g.
Initial cursor

```javascript
process.on('uncaughtException', (error, origin) => {
  log.error('uncaught exception', { origin }, error);
  newrelic.noticeError(error, {
    nodeProcessErrorType: 'uncaughtException',
    origin,
  });
```

init_selection = select inside quotes (exclusive)

```javascript
process.on('uncaughtException', (error, origin) => {
  log.error('uncaught exception', { origin }, error);
  newrelic.noticeError(error, {
    nodeProcessErrorType: 'uncaughtException',
```

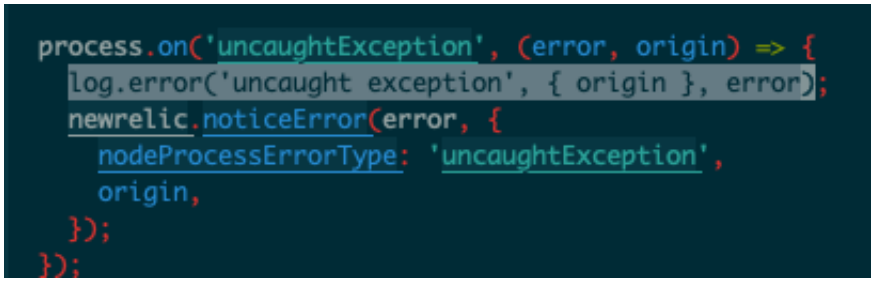
node_incremental = select quotes

```javascript
process.on('uncaughtException', (error, origin) => {
  log.error('uncaught exception', { origin }, error);
  newrelic.noticeError(error, {
    nodeProcessErrorType: 'uncaughtException',
    origin,
  });
```

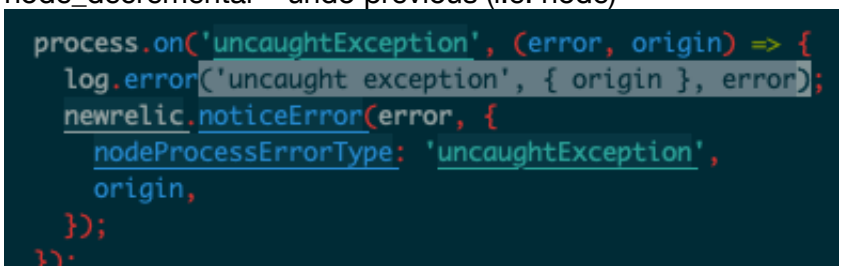
node_incremental = select inside brackets (inclusive)

```javascript
process.on('uncaughtException', (error, origin) => {
  log.error('uncaught exception', { origin }, error);
  newrelic.noticeError(error, {
    nodeProcessErrorType: 'uncaughtException',
    origin,
  });
```

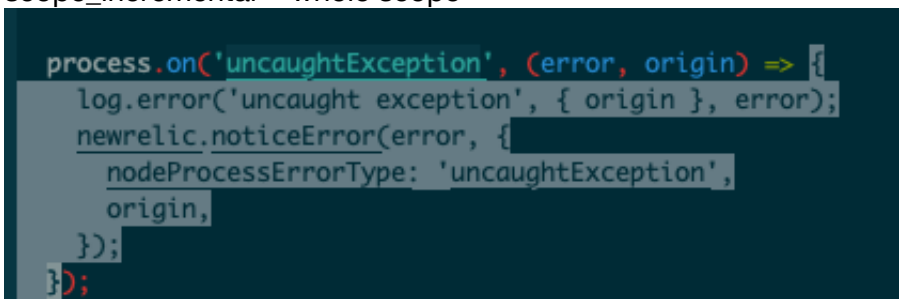
node_incremental = select whole line



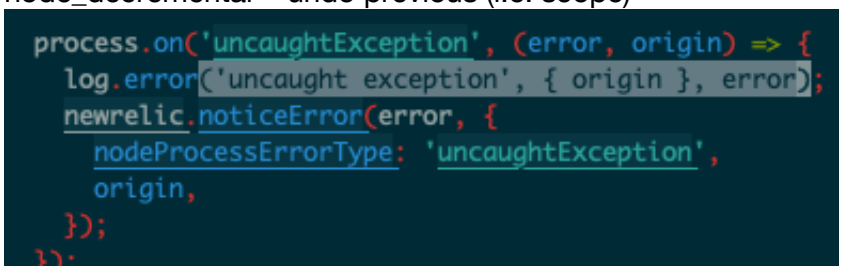
node_decremental = undo previous (i.e. node)



scope_incremental = whole scope



node_decremental = undo previous (i.e. scope)



[DEPRECATED neovim <= 0.11] Installation and configuration
This uses the old API
Old (<= neovim 0.11): https://github.com/nvim-treesitter/nvim-treesitter/tree/master
New (>= neovim 0.11): https://github.com/nvim-treesitter/nvim-treesitter/tree/main

```
"" vimplug
call plug#begin('~/.vim/plugged')
 Plug 'nvim-treesitter', { 'do': 'TSUpdate' }

 … other plugins …
lua <<EOF


"" fold settings
set foldmethod=expr
set foldexpr=nvim_treesitter#foldexpr()


"" treesitter configuration
require'nvim-treesitter.configs'.setup {
```

```
  -- one of "all", "maintained" (parsers with maintainers), or a
list of languages
  -- ensure_installed = "maintained",
  ensure_installed = {
    "bash",
    "c",
    "comment",
    "javascript",
    "json",
    "html",
    "python",
    "regex"
  },
  -- List of parsers to ignore installing
  ignore_install = {},

  -- highlight module
  highlight = {
    enable = true,
    -- enable = false,

    -- list of language that will be disabled
    disable = { "c", "rust" },

    -- Setting this to true will run `:h syntax` and tree-sitter at
the same time.
    -- Set this to `true` if you depend on 'syntax' being enabled
(like for indentation).
    -- Using this option may slow down your editor, and you may see
some duplicate highlights.
    -- Instead of true it can also be a list of languages
    additional_vim_regex_highlighting = false,
  },

  -- Incremental selection module
  incremental_selection = {
    enable = true,
    keymaps = {
      init_selection = "<leader>ii",
      node_incremental = "<leader>in",
      scope_incremental = "<leader>is",
      node_decremental = "<leader>id",
    }
  },


  -- indent module
  indent = {
    enable = true,
    -- enable = false
  }
}
EOF
```

Commands

```
" Help
:help nvim-treesitter

"" Info
" List supported and installed languages
TSInstallInfo

" List supported modules per language
TSModuleInfo

" Print effective config
TSConfigInfo


"" Installation
TSInstall "javascript"        " Install language
TSUpdate "javascript"
TSUninstall "javascript"
…


"" enable/disable
"
" Current buffer
TSBufEnable highlight        " Enable highlight module in current
buffer
TSBufDisable highlight       " Disable
TSBufToggle highlight        " Toggle

" New buffers
TSEnableAll highlight                   " Enable highlight module
for all languages
TSBufEnable highlight                   " TSEnableAll does not
enable on current buffer.
TSEnableAll highlight "javascript"      " Enable highlight module
for JS only
TSDisableAll highlight
TSToggle highlight
```

==========

Usage + troubleshooting

Issue: Error like "treesitter/highlighter: error executing lua: /usr/share/nvim/runtime/lua/vim/treesitter/query.lua:172"
Reference: https://github.com/nvim-treesitter/nvim-treesitter/issues/3091

Solution: reinstall treesitter

e.g.

```
# Delete
$ rm ~/.vim/plugged/nvim-treesitter
```

```
# Reinstall
$ nvim

:PlugInstall
```

Issue: [neovim >= 0.12 tree-sitter] Error like "Error during "tree-sitter build": vim/_core/system.lua:0: ENOENT: no such file or directory (cmd): 'tree-sitter'
Problem: Error trying to install a parser
e.g.

```
:TSInstall haskell
"tree-sitter build": vim/_core/system.lua:0: ENOENT: no such file
or directory (cmd): 'tree-sitter'
```

Solution: missing **tree-sitter-cli**
e.g.

```
$ brew install tree-sitter-cli
```

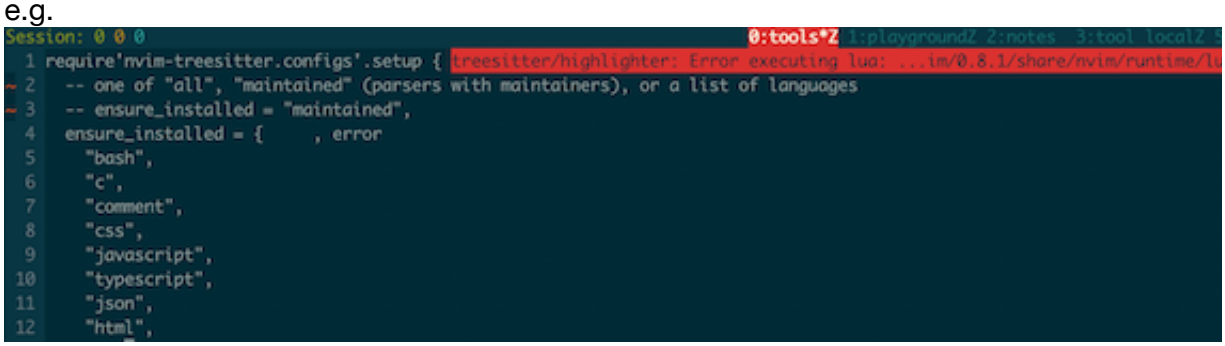
Issue: fold doesn't work
Issue: fold is still experimental, according to official doc

:help nvim_treesitter#foldexp

```
Note: This is highly experimental, and folding can break on some
types of
      edits. If you encounter such breakage, hitting `zx` should
fix folding.
      In any case, feel free to open an issue with the reproducing
steps.
```

Solution: reload using **zx**

[ERROR] treesitter highlighter error e.g. "query.lua: query : invalid node type at position <num>"
e.g.



Problem: **highlighter** is broken.
Usually just an outdated **parser**.

Solutions
Either
a) **update/reinstall** parser
b) uninstall + **disable** highlighter

*) **Solution 1**: update parser
e.g. update lua parser

Step 1: Update

```
-- a) update, OR
:TSUpdate lua

-- b) reinstall
:TSInstall! lua        -- force install, OR

:TSUninstall lua       -- uninstall + isntall
:TSInstall lua
```

Step 2) Verify no more error



*) **Solution 2**: uninstall and disable
e.g. disable lua

Step 1: Uninstall

```
:TSUninstall lua
```

Step 2: disable highlighter

```
require'nvim-treesitter.configs'.setup {
  …
  highlight = {
    enable = true,
    disable = { "c", "rust", "lua" },
    …
  },
 …
}
```

Step 3: verify no more error


Session: 0 0 0
0:tools*Z 1:playgroundZ 2:notes 3:tool
require'nvim-treesitter.configs'.setup {
-- one of "all", "maintained" (parsers with maintainers), or a list of languages
-- ensure_installed = "maintained",
ensure_installed = {
"bash",
"c",
"comment",
"css",
"javascript",
"typescript",
"json",
"html",
"python",
"regex",
"lua",